COMS 4701 AI Master Cheat Sheet

Comprehensive Review, Algorithms, Proofs, Examples & Exam Pro-Tips

1. WHAT IS AI & AGENTS?

AI aims to design **Intelligent Agents** that perceive environments and act to maximize success. **4 Schools of Thought:**

- *Thinking humanly*: Cognitive approach (modeling internal brain activities).
- *Acting humanly*: Turing Test (1950).
- *Thinking rationally*: Laws of thought/logic (computationally heavy).
- *Acting rationally*: **Our approach.** Acting to achieve the best expected outcome.

Rational Agents & Environments

Core Formula: *Agent = Architecture + Program.* **PEAS Specification**: **P**erformance (criteria for success), **E**nvironment (setting), **A**ctuators (mechanisms to act), **S**ensors (mechanisms to perceive). **Example:** Self-Driving Car: P(safety, time), E(roads), A(steering, brakes), S(cameras, GPS).

Environment Types:

- **Fully vs. Partially Observable**: Can sensors access the complete state?
- **Deterministic vs. Stochastic**: Is next state fully determined by current state + action? (If exceptions are only other agents, it's *Strategic*).
- **Static vs. Dynamic**: Does environment change while thinking? (*Semi-dynamic*: environment static, but score changes).
- **Discrete vs. Continuous**: Chess vs. Driving.
- **Single vs. Multi-agent**: Cooperative/competitive?
- **Known vs. Unknown**: Does agent know the laws of physics?

Agent Types & State Representations

- **Reflex Agents**: Act based on current state mapping.
- **Model-based**: Maintain internal model of how world evolves.
- **Goal-based**: Consider future impact to find sequence to goal.
- **Utility-based**: Maximize happiness / expected outcome.

State Representations: Atomic (black box, e.g., Search, MDP), **Factored** (attribute-value pairs, e.g., CSPs), **Structured** (relationships, e.g., FOL).

2. PROBLEM SOLVING & SEARCH

Map environment to structured math model:

1. **Initial State**: Starting configuration.
2. **State Space**: All reachable states.
3. **Action Space**: $Actions(s)$ returns executable actions.
4. **Transition Model**: $Results(s, a)$ returns resulting state.
5. **Goal Test**: Condition for winning.
6. **Path Cost**: Numeric cost for a path.

Tree vs. Graph Search: Tree search risks infinite loops due to repeated states. Graph search adds an **Explored Set** (Closed List) to prevent redundant work. **Space Regions**: Explored, Frontier (Open List), Unexplored.

Example: 3-Tower Hanoi: 3 pegs, 3 disks (S,M,L). Representation: 3 ordered sets. Initial: $\{1 : \{S, M, L\}, 2 : \{\}, 3 : \{\}\}$. States: $3^3 = 27$. Actions: Move top disk. Goal: $\{1 : \{\}, 2 : \{\}, 3 : \{S, M, L\}\}$. **Example: 8-Queen:** Place 8 queens so none attack. 1.8×10^{14} sequences. **8-Puzzle:** 3×3 grid, actions: up, down, left, right.

Exam Tip: ***Minimalism:*** *State should ONLY include what changes (agent pos, remaining food). Do NOT include static elements (maze walls).* ***Combinatorics:*** *Assigning N objects to M slots is often M^N.*

3. UNINFORMED SEARCH STRATEGIES

Search without heuristic guidance (brute force). **Variables:** b (branching factor), d (shallowest goal depth), m (max depth).

- **BFS (Breadth-First):** Queue (FIFO). Explores layer by layer. Complete: Yes. Optimal: Yes (equal step costs). Time/Space: $O(b^d)$ $(1+b+b^2+...+b^d)$. Memory is the biggest handicap.
- **DFS (Depth-First):** Stack (LIFO). Pursues branch to max depth. Complete: No (fails in infinite spaces). Optimal: No. Time: $O(b^m)$. Space: $O(bm)$. **Linear space complexity!** (Only stores a single path + unexpanded siblings).
- **DLS (Depth-Limited):** DFS with limit L. Time: $O(b^L)$, Space: $O(bL)$.
- **IDS (Iterative Deepening):** Repeated DLS with increasing limits. Complete & Optimal. Time: $O(b^d)$ $((d+1)b^0 + db^1 + ... + b^d)$. Space: $O(bd)$. **Theorem (Korf 1985):** Asymptotically optimal; wasted computation bounded by $C = (1 - 1/b)^{-2}$.
- **UCS (Uniform-Cost):** Priority Queue by $g(n)$. Expands lowest path cost. Complete & Optimal. Time/Space: $O(b^{C^*/\epsilon})$ where C^* is optimal cost, ϵ is min step cost. Effective depth is C^*/ϵ.

Frontier Shapes: BFS is a horizontal layer; DFS is unexplored siblings; UCS is jagged (various depths).

Exam Tip: ***Visit Order:*** *Exact order nodes are POPPED, not generated. Apply tie-breaking (U-R-D-L) strictly.* ***DFS Trick:*** *Push to stack in reverse priority so highest is on top.* ***IDS:*** *Always restart from root for each new limit L.* ***UCS/A* Trap:*** *If lower cost path found to node in PQ, update $g(n)$ AND parent.*

4. INFORMED SEARCH & HEURISTICS

Reduces blindness using intuition/domain knowledge ($h(n)$).

- **Greedy Best-First:** Expands $f(n) = h(n)$.
- **A* Search:** Minimizes total estimated cost $f(n) = g(n) + h(n)$. Complete & Optimal.

Heuristic Properties & Proofs

Admissibility (Optimistic): $0 \leq h(n) \leq h^*(n)$ (h^* is true optimal cost). Never overestimates. Guarantees optimality in **Tree Search**.

Proof of A Tree Optimality:*
Let G_o be optimal goal, G_s suboptimal goal. n is unexpanded node on path to G_o. 1. $f(G_s) = g(G_s)$, $f(G_o) = g(G_o)$ (goals $h = 0$).
2. G_s suboptimal $\implies f(G_s) > f(G_o)$.
3. $h(n) \leq h^*(n)$ (admissible).
4. $g(n) + h(n) \leq g(n) + h^*(n) = g(G_o) \implies f(n) \leq f(G_o)$.
5. Therefore, $f(G_s) > f(n)$. A* expands n before G_s.

Consistency (Monotonicity): $h(n) \leq c(n, a, n') + h(n')$. Satisfies triangle inequality. Guarantees optimality in **Graph Search**. Consistency $\implies$ Admissibility. $f(n)$ is monotonically non-decreasing.

Proof of A Graph Optimality:*
Assume A* reaches n' sub-optimally. 1. $f(n') \leq f(n)$ (n' expanded first).
2. $g(n') + h(n') \leq g(n) + h(n) \implies g(n') - g(n) \leq h(n) - h(n')$.
3. Suboptimal path $\implies g(n') > g(n) + c(n, a, n')$.
4. $c(n, a, n') < g(n') - g(n) \leq h(n) - h(n')$.
5. $c(n, a, n') < h(n) - h(n') \implies h(n) > c(n, a, n') + h(n')$.
6. Contradicts consistency definition!

Dominance: If $h_2(n) \geq h_1(n)$ $\forall n$, h_2 dominates h_1. A* with h_2 expands fewer/equal nodes. If h_1, h_2 admissible,

$\max(h_1, h_2)$ is admissible and dominates both. Crafted by **Relaxed Problems**.

Example: 8-Puzzle Relaxed: h_1 = misplaced tiles. h_2 = Manhattan distance. h_2 dominates h_1.

Exam Tip: ***Evaluating H manually:*** *1. Calculate TRUE h^* for all nodes first. 2. Admissibility: check $h \leq h^*$, one violation ruins it. 3. Consistency: check $h(start) - h(end) > edge_cost$; if true, inconsistent.*

5. LOCAL SEARCH

Used when state spaces are massive (10^{10} to 10^{100}) or pure optimization. $O(1)$ memory; maintains a single "current" state. **Landscape**: Global optimum, Local maximum (trap), Plateau (stagnation), Shoulder.

- **Hill Climbing (Steepest Ascent):** Greedy. $next = \arg\max f(s')$. Terminates at local optima. Complexity: $O(k \cdot |\mathcal{N}(s)|)$. *Variants:* Sideways moves (escape plateaus), Random-restart, Stochastic.
- **Local Beam Search:** Keeps k states. Generates all successors of k states, pools them, picks best k. Shares info but risks **Beam Concentration** (clustering at local optima). Stochastic beam selects probabilistically $P(n) = V(n)/\sum V(i)$.
- **Genetic Algorithms (GA):** Population-based stochastic optimization inspired by biological natural selection.
  - **Components:** *Population* (k individuals), *Genotype* (string representation), *Fitness Function* $f(s)$ (measures quality, maps Genotype $\to \mathbb{R}^+$).
  - **Selection:** *Fitness-Proportionate* (Roulette wheel, $P(select\ i) = f(s_i)/\sum_j f(s_j)$), *Tournament* (best from random subset), *Rank-based*.
  - **Crossover:** Combines parents. Types: Single-point, Uniform, Order-preserving. Prob $p_c \in [0.6, 0.9]$.
  - **Mutation:** Random alterations to maintain diversity. Types: Bit-flip, Swap. Prob $p_m \in [0.001, 0.01]$.
  - **Properties:** $O(n)$ memory, global search, slow but best for complex landscapes.
- **Simulated Annealing (SA):** Escapes local optima via probabilistic bad moves. Metropolis Criterion: If $\Delta E \leq 0$ (improvement), always accept. If $\Delta E > 0$, accept with $P = e^{-\Delta E/T}$. High T = exploration/random walk; Low T = exploitation/greedy. **Cooling:** Logarithmic $T = c/\log(1+t)$ (guarantees global but slow), Exponential $T = T_0\alpha^t$ (fast, $\alpha \in [0.8, 0.99]$), Linear $T = T_0 - \beta t$.

Example: 8-Queens GA: Genotype = string of 8 digits $\{1, \ldots, 8\}$. Position = column, Value = row (e.g., [3,2,7,4,8,5,5,2]). Fitness $f(s) = \binom{8}{2} - conflicts = 28 - conflicts$. Perfect solution $f = 28$. **Example: TSP SA:** Intractable for $n \geq 20$. State space $(n-1)!/2$. SA finds tour within 2-5% of optimal using 2-opt/3-opt swaps.

Exam Tip: ***SA Min vs Max:*** *Read carefully if minimizing (distance) or maximizing (profit). Adjust ΔE signs. Improving means negative Δ for distance, positive for profit.*

6. CONSTRAINT SATISFACTION (CSPS)

Goal: Find consistent assignments satisfying constraints. Cares about goal state, not path.

- **Variables** X: Discrete finite ($O(d^n)$ complete assignments), Discrete infinite (needs constraint language, $T_1 + d \leq T_2$), Continuous (linear programming).
- **Domains** D: Set of allowable values.
- **Constraints** C: Unary (single var), Binary (pairs/arcs), Global (Alldiff, 3+ vars), Preferences (Soft/costs).

Example: Map Coloring: Vars=Regions, Domains=Colors, Constraints=$SA \neq WA$. **Cryptarithmetic:** TWO+TWO=FOUR. `Alldiff` + addition logic. **Sudoku:** 81 vars, 27 Alldiff constraints.

Solving CSPs & Heuristics

Backtracking Search (BTS): Uninformed DFS, assigns one var at a time, checks constraints on the go to avoid conflicts. Time $O(d^n)$.

- **MRV (Min Remaining Values):** Pick var with fewest legal values left ("Pick the hardest").
- **LCV (Least Constraining Value):** Pick value ruling out fewest options for unassigned neighbors ("Pick likely to work").
- **Forward Checking (FC):** Assign value, remove conflicts from unassigned neighbors. **Domain Wipeout:** If any neighbor domain hits {}, backtrack immediately. Limitation: Does not check interactions between unassigned vars.

Exam Tip: ***Pure BTS:*** *Act dumb. Try values in exact numerical order. Don't skip bad values using intuition; mark them "FAIL".* ***FC:*** *Stop expanding node instantly on wipeout.*

Constraint Propagation & Structure

Node Consistency: Values satisfy unary constraints.
Arc Consistency (AC-3): $X \to Y$ consistent if every $x \in D_X$ has an allowed $y \in D_Y$. Algorithm: If `REVISE(X,Y)` shrinks D_X, **re-queue all neighbors pointing to** X ($Z \to X$). Complexity: $O(n^2d^3)$.
Problem Structure:

- **Independent Subproblems:** Decomposing to size c reduces time from $O(d^n)$ to $O((n/c)d^c)$.
- **Tree-Structured CSPs:** Any two vars connected by one path. Linear time $O(nd^2)$. Steps: 1. Topological Sort (root to leaves). 2. Make Directed Arc Consistent (DAC) bottom-up (n down to 2). 3. Assign values top-down (1 to n) with zero backtracking!
- **Nearly Tree-Structured:** Assign fixed value to a variable (Cutset conditioning) to prune graph into a tree.